Object Oriented Programming (CSEG1044)
Unit 06 – Lecture 06 Notes
Testing + Build/Artifacts + Final Demo

Tofik Ali

February 27, 2026

Contents

1 Lecture Overview **1**

2 Syllabus Alignment (Unit 06) **2**

3 Learning Outcomes **2**

4 Key Terms (Quick) **2**

5 Detailed Notes **2**
5.1 What should you test? (capstone strategy) . . . 2
5.2 JUnit 5 basics (how a unit test looks) . . . 3
5.3 Design for testability: depend on interfaces . . . 3
5.4 Integration test checklist (DAO + DB) . . . 3
5.5 Build tool + packaging (Maven/Gradle idea) . . . 4

6 Worked Example: unit-test a BookingService without DB **4**
6.1 Repository interface + in-memory fake . . . 4
6.2 Service (rules) + tests . . . 5

7 Deliverable-5 checklist (final submission) **6**

8 Final Demo Script (suggested) **6**

9 Checkpoint Questions (with sample answers) **6**

1 Lecture Overview

This is the final capstone lecture. Your goal is not to add new features endlessly — your goal is to make the project **demo-ready**:

- core flows work reliably,
- mistakes are handled gracefully (validation + errors),

- the project builds into a runnable artifact,
- you have a clear demo script and evidence of teamwork.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

2 Syllabus Alignment (Unit 06)

Unit 06: testing (JUnit), build tool + packaging, final demo/viva

3 Learning Outcomes

After this lecture, you should be able to:

- Write unit tests for your service/business rules using JUnit 5.
- Perform an integration test checklist for DAO + DB.
- Package the project as a runnable artifact (Jar) using Maven/Gradle (or a simple build script).
- Complete Deliverable-5: final demo/viva + release artifact + short report.

4 Key Terms (Quick)

- **Unit test**: tests one class/function in isolation (fast, deterministic).
- **Integration test**: tests multiple parts together (DAO + DB, UI + service).
- **Test pyramid**: many unit tests, fewer integration tests, minimal UI/manual tests.
- **Assertion**: check that expected result is correct (`assertEquals`, `assertThrows`).
- **Fixture**: setup data/objects for tests (`@BeforeEach`).
- **Artifact**: build output you can run/distribute (Jar).
- **Release**: versioned snapshot of your project (tagged commit + artifact).

5 Detailed Notes

5.1 What should you test? (capstone strategy)

Best unit-test targets:

- service-layer validation and business rules ("no overlap", "capacity > 0")
- small pure functions (parsers, formatters)

Harder to unit test (do minimal/manual):

- Swing UI (usually tested manually in this course)
- DAO mapping (better as integration tests with a test database)

Why? Service rules are where bugs hurt the most, and they are easy to test quickly without a GUI or DB.

5.2 JUnit 5 basics (how a unit test looks)

JUnit tests have a simple structure:

- Arrange: prepare objects
- Act: call the method
- Assert: check result/exception

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class SimpleTest {
    @Test
    void endMustBeAfterStart() {
        ValidationException ex = assertThrows(
            ValidationException.class,
            () -> BookingRules.validateTime(10, 10)
        );
        assertTrue(ex.getMessage().contains("after"));
    }
}
```

Tip: test names should describe behavior: `rejectsEndBeforeStart`.

5.3 Design for testability: depend on interfaces

If your service depends directly on a JDBC DAO, tests become slow and fragile. Instead:

- define an interface (contract), e.g., `BookingRepository`,
- implement JDBC repository for real app,
- implement an in-memory fake for unit tests.

5.4 Integration test checklist (DAO + DB)

Use a separate test DB/schema (never your real production data).

1. Run `schema.sql` on a clean database.
2. Run DAO insert for one entity.
3. Run DAO list/find and verify data is correct.

4. Run update and verify changes.
5. Run delete and verify row is gone.
6. Test one "bad input" case (FK constraint violation, duplicate UNIQUE).

5.5 Build tool + packaging (Maven/Gradle idea)

Why use a build tool?

- consistent compilation and testing for everyone in team,
- dependency management (JDBC driver, JUnit),
- easy packaging into a Jar.

Maven (common commands):

- `mvn test` — run unit tests
- `mvn package` — build Jar in `target/`

Gradle (common commands):

- `gradle test`
- `gradle build`

Packaging note: if your Jar needs external libraries (like MySQL connector), you either run with a classpath, or build a "fat Jar" that bundles dependencies.

Sources: Git SCM: *Pro Git* (2e) (version control workflow), accessed February 27, 2026.; GitHub Docs (repositories, issues, pull requests), accessed February 27, 2026.; Oracle Java Tutorials: Swing + JDBC (accessed February 27, 2026).; JUnit 5 User Guide (accessed February 27, 2026).; Apache Maven Project: Getting Started (accessed February 27, 2026).; Gradle User Manual: Getting Started (accessed February 27, 2026).; UML class diagrams: UML 2.x overview/spec (accessed February 27, 2026).

6 Worked Example: unit-test a BookingService without DB

This example shows the key pattern for capstone testing: service depends on an interface, not JDBC.

6.1 Repository interface + in-memory fake

```
import java.util.*;

interface BookingRepository {
    boolean hasOverlap(int resourceId, long start, long end);
    void save(Booking b);
    int count();
}

class InMemoryBookingRepository implements BookingRepository {
    private final List<Booking> data = new ArrayList<>();
```

```java
    public boolean hasOverlap(int resourceId, long start, long end) {
        for (Booking b : data) {
            if (b.getResourceId() != resourceId) continue;
            boolean overlaps = start < b.getEnd() && end > b.getStart();
            if (overlaps) return true;
        }
        return false;
    }

    public void save(Booking b) { data.add(b); }
    public int count() { return data.size(); }
}
```

6.2 Service (rules) + tests

```java
class BookingService {
    private final BookingRepository repo;
    BookingService(BookingRepository repo) { this.repo = repo; }

    public void createBooking(int resourceId, String user, long start, long end) {
        if (user == null || user.trim().isEmpty())
            throw new ValidationException("User is required.");
        if (end <= start)
            throw new ValidationException("End must be after start.");
        if (repo.hasOverlap(resourceId, start, end))
            throw new ValidationException("Booking overlaps existing booking.");
        repo.save(new Booking(resourceId, user.trim(), start, end));
    }
}
```

```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.*;

class BookingServiceTest {
    private InMemoryBookingRepository repo;
    private BookingService service;

    @BeforeEach
    void setup() {
        repo = new InMemoryBookingRepository();
        service = new BookingService(repo);
    }

    @Test
    void rejectsEndBeforeStart() {
        assertThrows(ValidationException.class,
            () -> service.createBooking(1, "A", 10, 10));
    }

    @Test
    void rejectsOverlap() {
        service.createBooking(1, "A", 10, 20);
```

```
        assertThrows(ValidationException.class,
            () -> service.createBooking(1, "B", 15, 25));
    }

    @Test
    void acceptsNonOverlap() {
        assertDoesNotThrow(() -> service.createBooking(1, "A", 10, 20));
        assertDoesNotThrow(() -> service.createBooking(1, "B", 21, 30));
        assertEquals(2, repo.count());
    }
}
```

7 Deliverable-5 checklist (final submission)

- Runnable artifact: Jar (plus any config instructions).
- `schema.sql` + DB setup instructions.
- Basic unit tests for service rules (JUnit) + evidence they pass.
- README: how to run, demo steps, team members + responsibilities.
- Short report (1–2 pages): design summary + what was implemented.

8 Final Demo Script (suggested)

1. Start DB and ensure schema exists.
2. Run the application.
3. Show one end-to-end use case: Add Resource → verify in list.
4. Show booking flow: create booking → list bookings.
5. Show validation: attempt an invalid booking (end before start) and show message.
6. Briefly show code structure: UI/service/DAO packages.
7. Show tests running (optional but impressive): `mvn test`.

9 Checkpoint Questions (with sample answers)

Q1. Why unit test service logic instead of Swing UI?

Sample answer: Service logic is easy to test quickly and deterministically; UI tests are slower and fragile.

Q2. What does `assertThrows` test?

Sample answer: It verifies that a method call throws an expected exception for an invalid case.

Q3. Why use a build tool?

Sample answer: It standardizes builds/tests, manages dependencies, and produces a runnable artifact reliably for the team.